


**Raspodijeljeni sustavi, Predavanje 02**

# Arhitekture i granice usluga

**Fakultet informatike, Sveučilište Jurja Dobrile u Puli**

izv. prof. dr. sc. Nikola Tanković

## 1 Ishodi učenja

Nakon ovog predavanja trebali biste moći:

1. Razlikovati logičku arhitekturu sustava (komponente, njihove odgovornosti i veze) od fizičkog smještaja tih komponenti na računala.
2. Opisati arhitekturu klijent–poslužitelj, višeslojnu arhitekturu te vertikalnu i horizontalnu raspodjelu.
3. Prepoznati granicu usluge i procijeniti njezinu cijenu: dodatnu latenciju, manju dostupnost lanca poziva i podatke koje treba uskladiti preko granice.
4. Usporediti sustave klijent–poslužitelj i P2P te objasniti kada je središnji koordinator koristan.
5. Odrediti granice razmještaja (proces, kontejner, računalo, podatkovni centar) i objasniti što unutar njih dijeli ispad.

**Predznanje:** predavanje 01, osobito raspodijeljeni sustav, djelomični ispad, istek vremena čekanja (timeout), ponovni pokušaj (retry) i invarijanta.

**Opseg predavanja:** ovdje biramo *gdje* su komponente i *kako* su povezane. Kako se poruke točno šalju (RPC, serijalizacija, istek čekanja) obrađuje predavanje 03, redovi poruka predavanje 10, a replikacija podataka predavanje 06. Portal za predaju projekata i dalje je zamišljeni primjer, a ne stvarni sustav sveučilišta.

## 2 Primjer: portal pred rok

U predavanju 01 pratili smo jednu predaju projekta kroz portal. Sada portal raste. Nastavnici traže novu funkciju: nakon prijenosa portal treba **provjeriti projekt**. Provjera

raspakira ZIP datoteku s Unity projektom, provjeri očekivanu strukturu mapa i skenira datoteke na zlonamjerni kod. Provjera intenzivno koristi procesor i traje znatno dulje od običnog zahtjeva.

Prva verzija portala je jedan program: prijava korisnika, predaja, rokovi i nova provjera nalaze se u istom procesu i dijele jednu bazu. Večer prije roka stotine studenata predaju projekte gotovo istodobno. Provjere zauzmu procesor pa se usporavaju i dijelovi koji s provjerom nemaju veze: prijava i preuzimanje radova za nastavnike. Zatim pogreška u biblioteci za raspakiravanje sruši proces. Budući da je sve u istom procesu, nitko ne može predati projekt dok se portal ponovno ne pokrene. Popravak provjere traži novo izdanje cijelog portala.

**Pitanje za ovo predavanje:** kamo staviti provjeru projekta? U isti program kao ostatak portala, u zasebnu uslugu ili „ovisi”? Da bismo odgovorili, trebamo rječnik za opis arhitekture i način da procijenimo što pojedina podjela košta.

### 2.1 Za početak: računi 1, 2, 3, …

Domaća zadaća iz predavanja 01 tražila je slijedne brojeve računa koje izdaje više poslužitelja, bez ponavljanja i bez preskakanja. Najjednostavnije rješenje je jedan **središnji brojač**: svaki poslužitelj od njega traži sljedeći broj. To je arhitekturna odluka. Jedna komponenta dobiva odgovornost i jedini drži stanje (posljednji izdani broj), a ostale joj šalju zahtjeve.

Ta odluka ima posljedice. Slijed je lako očuvati jer o broju odlučuje jedno mjesto. Ali brojač postaje usko grlo i jedina točka ispada: kada je nedostupan, nijedan poslužitelj ne može izdati račun. Ako poslužitelj uzme broj 42 i sruši se prije spremanja računa, nastaje rupa u slijedu, osim ako se broj dodjeljuje tek pri spremanju računa. Ako blagajni istekne vrijeme čekanja pa pokuša ponovno, potreban je identifikator namjeravanog računa kako ponovni pokušaj ne bi stvorio dva računa. U ovom predavanju naučit ćemo takve odluke imenovati i usporediti.

## 3 Što je arhitektura sustava?

Kada kažemo „arhitektura portala”, mislimo na dvije povezane stvari. Prva je **logička organizacija**: koje komponente postoje, za što je koja odgovorna i koje poruke razmjenjuju. Druga je **fizički smještaj**: na kojim se računalima i u kojim procesima te komponente izvršavaju. Van Steen i Tanenbaum razlikuju organizaciju softverskih komponenti od njihova stvarnog smještaja na računala [1, pogl. 2].

**Definicija 2.1**

**Arhitektura raspodijeljenog sustava (system architecture)** opisuje komponente sustava, njihove odgovornosti i veze te smještaj komponenti na računala i u procese.

Razlika je važna jer ista logička arhitektura može imati vrlo različita svojstva ovisno o smještaju. Dvije komponente u istom procesu pozivaju se lokalno i ruše se zajedno. Iste komponente na dva računala razmjenjuju poruke preko mreže i mogu se srušiti neovisno. U predavanju 01 rekli smo da dijagram s četiri procesa ne znači četiri računala; ovdje to pravilo pretvaramo u alat za odlučivanje.

# 4 Klijent–poslužitelj

Najčešći oblik organizacije je **klijent–poslužitelj**. Poslužitelj nudi uslugu i čeka zahtjeve. Klijent šalje zahtjev i čeka odgovor. U portalu je preglednik klijent, a usluga predaje poslužitelj [1, pogl. 2].

**Definicija 2.2**

U arhitekturi **klijent–poslužitelj (client–server)** poslužitelj nudi uslugu i čeka zahtjeve, a klijent šalje zahtjev i čeka odgovor.

Klijent i poslužitelj su **uloge** u jednoj razmjeni, a ne vrste računala. U predavanju 01 vidjeli smo da je usluga predaje poslužitelj za preglednik, ali klijent za uslugu zadataka. Središnji brojač računa iz domaće zadaće također je poslužitelj, a poslužitelji koji izdaju račune njegovi su klijenti.

Prednost ove organizacije je jednostavnost: stanje i pravila nalaze se na poznatom mjestu, a klijenti ne moraju znati jedni za druge. Nedostatak je ovisnost o poslužitelju. Ako poslužitelj ne radi ili ne stigne obraditi sve zahtjeve, klijenti nemaju alternativu.

## 4.1 Slojevi i njihov smještaj

Aplikacije se često dijele na tri logička sloja: **korisničko sučelje**, **aplikacijsku logiku** i **podatke** [1, pogl. 2]. U portalu sučelje prikazuje obrazac za predaju, aplikacijska logika provjerava pravila zadatka i rokove, a sloj podataka čuva zapise i datoteke.

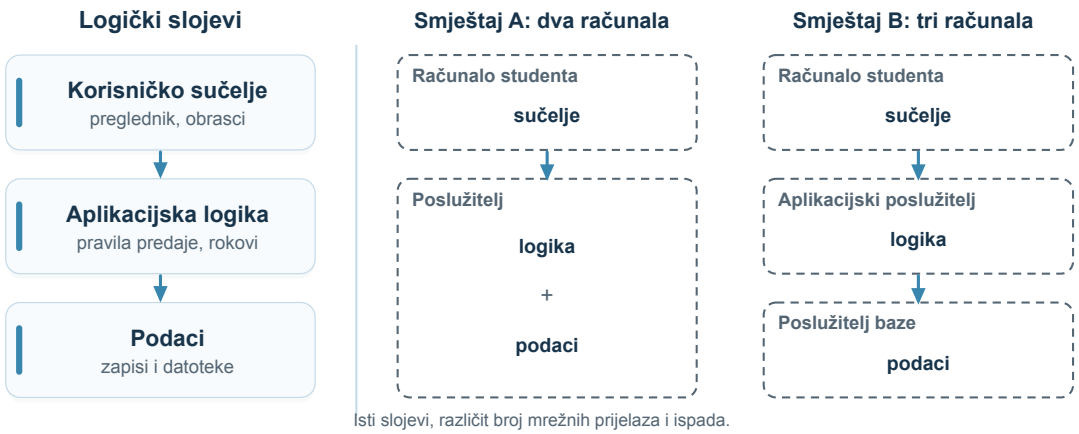



Figure 1: Isti logički slojevi mogu se smjestiti na dva ili tri računala. Svaka granica između računala dodaje mrežni prijelaz i novi mogući ispad. Strelica pokazuje od pozivatelja prema komponenti koju treba.

Slojevi su logička podjela; broj računala je odluka o smještaju. U **dvoslojnom** smještaju (two-tier) sučelje radi na računalu studenta, a logika i podaci na jednom poslužitelju. U **troslojnom** smještaju (three-tier) logika i baza nalaze se na zasebnim poslužiteljima. Treće računalo omogućuje da bazu skaliramo i održavamo zasebno, ali dodaje mrežni prijelaz u svaki zahtjev koji treba podatke i komponentu koja može zasebno prestati raditi.

## 4.2 Vertikalna i horizontalna raspodjela

Postoje dva osnovna načina da posao rasporedimo na više računala [1, pogl. 2]:

- **Vertikalna raspodjela** smješta *različite funkcije* na različita računala: sučelje na jedno, uslugu predaje na drugo, bazu na treće. Svako računalo radi nešto drugo.
- **Horizontalna raspodjela** pokreće *više kopija iste funkcije*, takozvanih **replika** (replicas). Raspoređivač opterećenja (load balancer) šalje svaki zahtjev jednoj od njih.

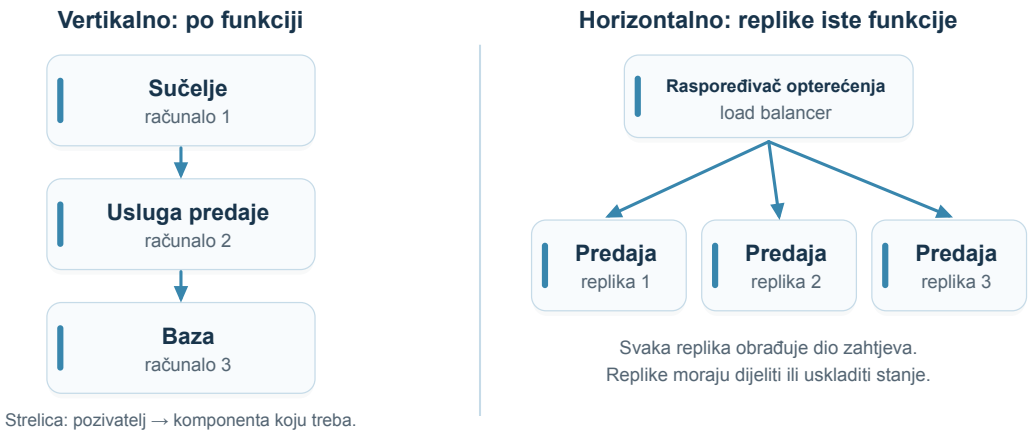



Figure 2: Vertikalna raspodjela dijeli sustav po funkciji, a horizontalna pokreće replike iste funkcije. Strelica pokazuje od pozivatelja prema komponenti koju treba.

Horizontalna raspodjela odgovara horizontalnom skaliranju (scale out) iz predavanja 01. Ona povećava kapacitet i omogućuje da sustav nastavi raditi kada jedna replika ispadne, ali samo ako replike ne drže stanje koje druge ne vide. Ako replika 1 pamti u memoriji da je predaja u tijeku, a sljedeći zahtjev istog studenta stigne na repliku 2, replika 2 to ne zna. Zato replike obično drže stanje u zajedničkoj bazi ili ga moraju uskladiti; to je tema predavanja 06.

# 5 Monolit i usluge

## 5.1 Monolit

Prva verzija našeg portala je **monolit** (monolith): cijela aplikacija izvršava se kao jedan proces i izdaje se kao jedna cjelina. Moduli se pozivaju lokalnim pozivima funkcija i dijele jednu bazu.

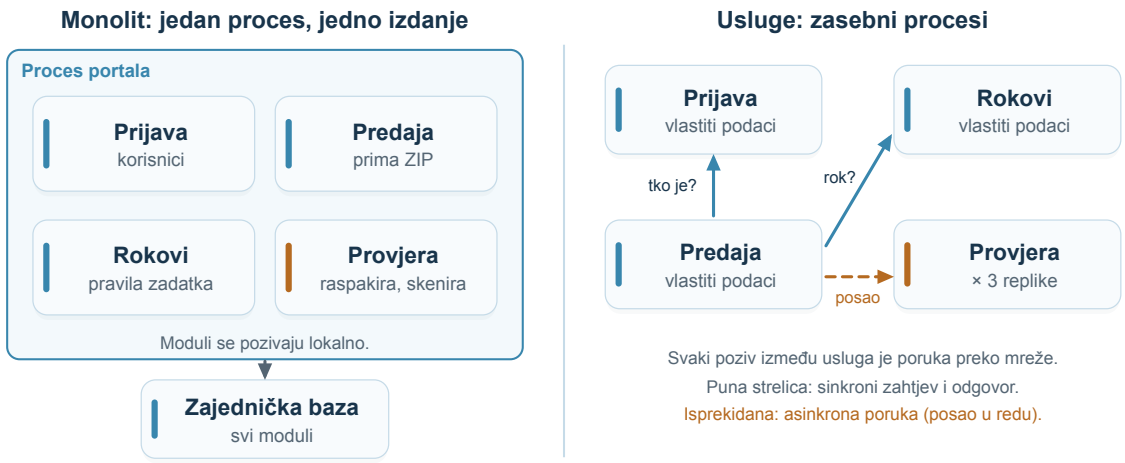



Figure 3: Lijevo: monolit, u kojemu su svi moduli u istom procesu i dijele bazu. Desno: iste funkcije kao zasebne usluge s vlastitim podacima. Puna strelica je sinkroni zahtjev s odgovorom; isprekidana je asinkrona poruka, na primjer posao u redu.

Monolit ima stvarne prednosti. Lokalni poziv je brz i ne može se izgubiti u mreži. Jedna transakcija u bazi može obuhvatiti sve module. Razvoj, testiranje i pokretanje su jednostavni. Za mnoge sustave, osobito na početku, monolit je razuman izbor.

Problemi nastaju kada dijelovi sustava imaju različite zahtjeve. U našem primjeru provjera troši procesor koji trebaju i prijava i preuzimanje, pa se ne može skalirati zasebno: da dobijemo više provjera, moramo pokrenuti više kopija cijelog portala. Pogreška u jednom modulu ruši cijeli proces. Promjena jednog modula traži novo izdanje svega. Svi moduli dijele istu **granicu ispada**.

## 5.2 Usluga i granica usluge

Alternativa je podijeliti aplikaciju na **usluge** (services). Lewis i Fowler opisuju stil mikroservisa kao razvoj jedne aplikacije kao skupa malih usluga od kojih se svaka izvršava u vlastitom procesu, komunicira laganim mehanizmima, organizirana je oko poslovne sposobnosti i izdaje se neovisno; svaka usluga upravlja vlastitim podacima [2].

**Definicija 2.3**

**Usluga (service)** je komponenta koja se izvršava zasebno, preko definiranog sučelja nudi skup povezanih funkcija i upravlja vlastitim podacima.

**Definicija 2.4**

**Granica usluge (service boundary)** odvaja uslugu od ostatka sustava. Sve što prelazi granicu postaje poruka preko sučelja usluge.

Granica usluge nije samo crta na dijagramu. Ona određuje tko smije mijenjati koje podatke i koji poziv postaje mrežna poruka. Kada usluga predaje treba rok, više ne čita varijablu ni tablicu modula rokova; šalje zahtjev usluzi rokova i čeka odgovor. Sve što smo u predavanju 01 rekli o udaljenim zahtjevima, djelomičnim ispadima i isteku čekanja sada vrijedi za taj poziv.

### 5.3 Gdje povući granicu?

Dobra granica okuplja odgovornosti koje se mijenjaju zajedno i razdvaja one koje imaju različite zahtjeve. Pri odluci pomažu sljedeća pitanja:

- **Povezane odgovornosti:** obavlja li usluga jednu prepoznatljivu poslovnu funkciju (rokovi, predaja, provjera)?
- **Vlastiti podaci:** postoje li podaci kojima upravlja samo ova usluga?
- **Različiti resursi i skaliranje:** treba li ovaj dio drukčije računalne resurse ili drukčiji broj replika od ostatka?
- **Izolacija ispada:** smije li ostatak sustava raditi dok ovaj dio ne radi?
- **Neovisno izdavanje:** mijenja li se ovaj dio drukčijim tempom ili ga razvija drugi tim?

Za provjeru projekta odgovori su uglavnom „da”: treba drukčije resurse, smije privremeno kasniti, a njezina pogreška ne smije srušiti predaju. Za modul koji samo formatira datume odgovori su „ne”; zasebna usluga donijela bi samo troškove.

Upozoravajući znak je granica preko koje dvije usluge neprestano razgovaraju. Ako usluga predaje za svaki korak mora pitati uslugu rokova, možda smo razdvojili odgovornosti koje pripadaju zajedno.

## 6 Što granica košta

Svaka granica usluge pretvara lokalne pozive u mrežne poruke. Cijenu procjenjujemo u tri dimenzije: vrijeme, dostupnost i podaci.

### 6.1 Latencija i razgovorljiva sučelja

Lokalni pristup memoriji i mrežni zahtjev razlikuju se za nekoliko redova veličine. U poznatoj tablici okvirnih vremena Jeffa Deana iz 2009. referenca na glavnu memoriju traje oko 100 ns, a povratno putovanje poruke unutar istog podatkovnog centra oko 500 µs, dakle oko 5000 puta dulje [6]. Te su brojke stare i služe samo za procjenu reda veličine, ali odnos ostaje: poziv preko granice usluge mnogo je skuplji od poziva unutar procesa.

Razlika postaje važna kod **razgovorljivog sučelja** (chatty interface). Nastavnik otvori popis od 200 predaja. Usluga predaje za svaku predaju zasebno pita uslugu korisnika za ime studenta. Ako jedan takav zahtjev s obradom traje 2 ms, 200 uzastopnih zahtjeva traje

oko 400 ms prije nego što stranica išta prikaže, a svaki od njih može isteći ili se izgubiti. Bolje je jednim zahtjevom tražiti imena za svih 200 studenata ili uz predaju čuvati kopiju imena. Kopija štedi pozive, ali može zastarjeti ako student promijeni ime; to je kompromis, a ne besplatno rješenje.

## 6.2 Dostupnost lanca poziva

Kada jedan korisnički zahtjev treba sinkroni odgovor nekoliko usluga redom, dostupnost cijelog lanca manja je od dostupnosti svake pojedine usluge.

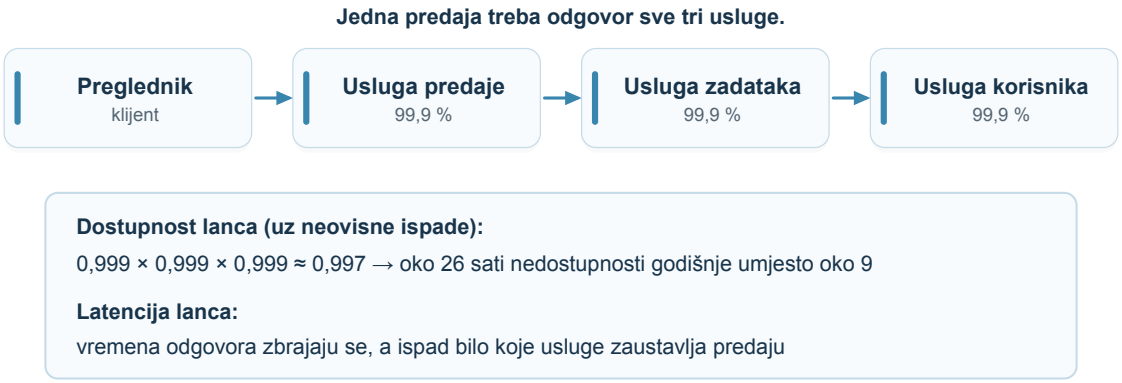



Figure 4: Predaja treba odgovor triju usluga. Uz pretpostavku neovisnih ispada dostupnosti se množe, a vremena odgovora zbrajaju.

Pretpostavimo da je svaka od triju usluga dostupna 99,9 % vremena i da ispadaju neovisno. Predaja uspijeva samo kada rade sve tri: $0{,}999^3 \approx 0{,}997$. Jedna usluga nedostupna 0,1 % vremena miruje oko 8,8 sati godišnje; lanac od tri miruje oko 26 sati. Pretpostavka neovisnosti je važna: ako sve tri usluge rade na istom računalu, ispad tog računala zaustavlja sve tri istodobno.

Latencije se u lancu zbrajaju. Ako svaka usluga uz to postavlja vlastiti istek čekanja i ponovni pokušaj, jedan spori dio može izazvati niz isteka čekanja i ponovnih pokušaja kroz cijeli lanac. Zato je korisno pitati treba li zahtjev doista *sinkroni* odgovor ili može nastaviti dok druga usluga posao obavi kasnije.

## 6.3 Podaci i pravila preko granice

Usluga upravlja vlastitim podacima [2]. Drugim uslugama ih daje samo preko sučelja. To znači da pravilo koje obuhvaća podatke dviju usluga više nije provjera unutar jedne transakcije u bazi. Pravilo „predaja se prihvaća samo prije roka” sada povezuje uslugu rokova i uslugu predaje. Usluga predaje mora pitati za rok ili čuvati njegovu kopiju i znati što učiniti ako se rok u međuvremenu promijeni. Kako takva pravila čuvati preko više usluga, tema je predavanja 09.

Primamljiv prečac je da sve usluge ostanu spojene na **zajedničku bazu**. Tada promjena strukture jedne tablice može srušiti više usluga, a usluge se moraju izdavati zajedno. Takav sustav ima troškove raspodijeljenog sustava (mrežne pozive, djelomične ispade), a zadržava ograničenja monolita (zajedničko izdanje, zajednička sudbina). Takav ishod nazivamo **raspodijeljeni monolit** (distributed monolith).

# 7 P2P sustavi

U arhitekturi klijent–poslužitelj posao i stanje koncentrirani su na poslužitelju. U **P2P sustavu** (peer-to-peer) sudionici su ravnopravni: svaki istodobno nudi i koristi resurse [1, pogl. 2].

**Definicija 2.5**

U **P2P sustavu (peer-to-peer)** ravnopravni sudionici istodobno nude i koriste resurse te glavni posao obavljaju bez središnjeg poslužitelja.

Kada poslužitelj šalje veliku datoteku tisućama klijenata, cijeli promet prolazi kroz njegovu vezu. U P2P sustavu svaki sudionik koji je preuzeo dio datoteke može taj dio slati drugima, pa kapacitet raste s brojem sudionika.

## 7.1 Primjer: BitTorrent

BitTorrent dijeli datoteku na dijelove (pieces) jednake veličine. Datoteka s metapodacima (`.torrent`) sadrži adresu trackera i SHA-1 sažetak svakog dijela. **Tracker** je poslužitelj koji sudionicima vraća popis drugih sudionika iste datoteke. Sudionici zatim izravno razmjenjuju dijelove, a svaki preuzeti dio provjeravaju prema sažetku prije nego što ga ponude drugima [3].

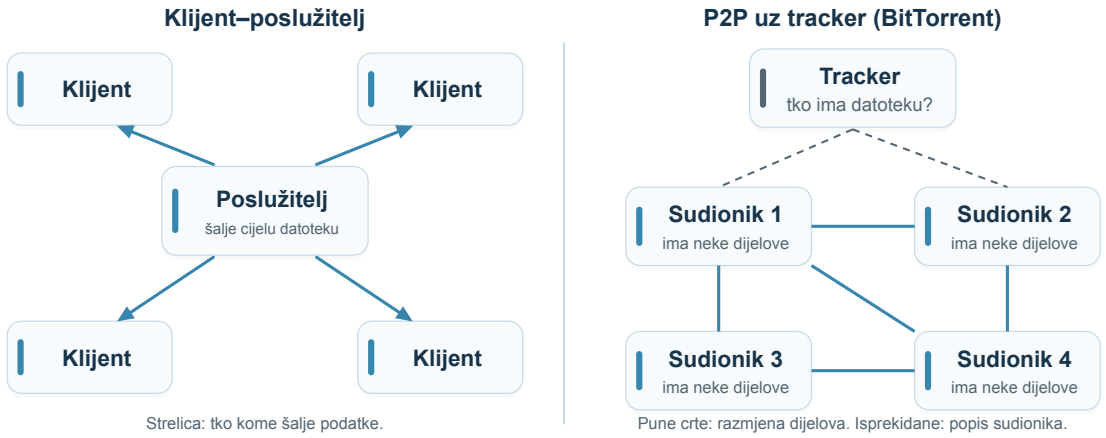



Figure 5: Lijevo poslužitelj sam šalje cijelu datoteku svakom klijentu. Desno tracker samo vraća popis sudionika, a sudionici izravno razmjenjuju dijelove datoteke.

Ovaj BitTorrent je **hibridan**: podaci putuju izravno između sudionika, ali popis sudionika daje središnji tracker. Proširenje BitTorrenta s raspodijeljenom tablicom sažetaka (distributed hash table, DHT) uklanja i tracker: sudionici pohranjuju kontakte drugih sudionika pod sažetkom datoteke pa, prema specifikaciji, „svaki sudionik postaje tracker" [4]. DHT je primjer **strukturiranog** P2P sustava, u kojemu pravilo određuje koji je sudionik zadužen za koji ključ. U **nestrukturiranim** P2P sustavima sudionik ne zna gdje se podatak nalazi pa upit prosljeđuje susjedima.

P2P ima i cijenu. Sudionici dolaze i odlaze kad žele; tu pojavu nazivamo **izmjena sudionika** (churn). Podatak je dostupan samo dok ga netko nudi. Sudionicima se ne može unaprijed vjerovati, zbog čega BitTorrent provjerava svaki dio prema sažetku. Pronalaženje podatka bez središnjeg popisa traži dodatne poruke.

### 7.2 Može li portal biti P2P?

Za službenu predaju odgovor je uglavnom ne. Nastavnik mora moći preuzeti projekt i kada je student isključio računalo. Netko mora jamčiti da je predaja zabilježena prije roka, a sudionici međusobno nisu pouzdani svjedoci. Takvo jamstvo traži **pouzdano mjesto odlučivanja**, što je razlog postojanja poslužitelja.

P2P je, međutim, koristan za druge dijelove sličnog sustava. Ako 200 studenata u laboratoriju treba preuzeti skup podataka od 50 GB za vježbu, računala u laboratoriju mogu međusobno dijeliti dijelove umjesto da svi preuzimaju sa sveučilišnog poslužitelja.

## 8 Klaster s koordinatorom

Mnogi sustavi za obradu podataka koriste srednji put: mnogo računala obavlja posao, a jedan proces ga koordinira. U Apache Sparku aplikaciju koordinira **driver**. Upravitelj klas-

tera (cluster manager) dodjeljuje resurse, a na računalima klastera pokreću se **executori**, procesi koji izvršavaju zadatke i čuvaju podatke aplikacije. Driver šalje zadatke executorima i mora biti mrežno dostupan executorima tijekom cijelog rada aplikacije; dokumentacija preporučuje da radi blizu radnih čvorova, po mogućnosti u istoj lokalnoj mreži [5].

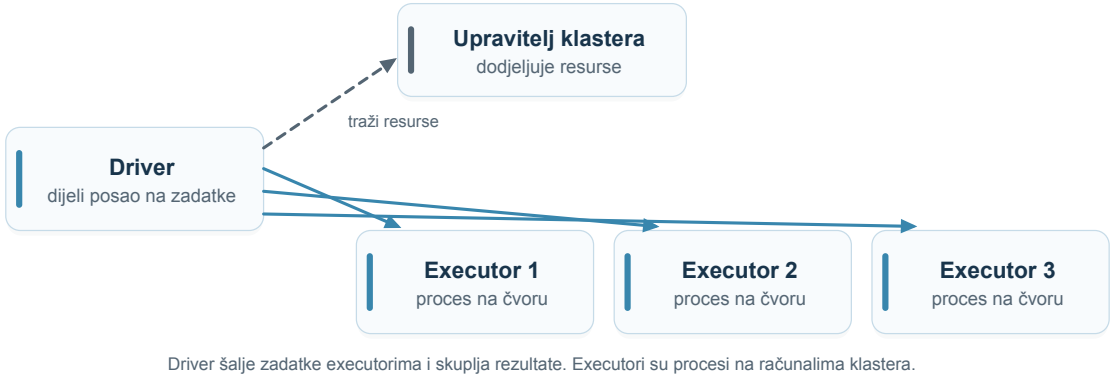



Figure 6: Driver dijeli posao na zadatke i šalje ih executorima. Upravitelj klastera dodjeljuje resurse. Strelica pokazuje od pozivatelja prema komponenti kojoj šalje zahtjev.

To je unutarnja arhitektura klijent–poslužitelj s jednim koordinatorom. Koordinator pojednostavljuje odlučivanje: jedan proces zna koji su zadaci gotovi. Zato je i on posebno osjetljiv dio sustava. Ova arhitektura odgovara jednom od dvaju tipičnih tipova projekta iz uvodnog predavanja: raspodijeljenoj obradi podataka, u kojoj se posao dijeli na procese, a rezultati objedinjuju.

## 9 Granice razmještaja

Logička arhitektura kaže koje usluge postoje. Odluka o smještaju kaže koje se od njih pokreću, zaustavljaju i ruše zajedno.

**Definicija 2.6**

**Granica razmještaja (deployment boundary)** obuhvaća komponente koje se pokreću, zaustavljaju i mogu srušiti zajedno, na primjer proces, kontejner, računalo ili podatkovni centar.

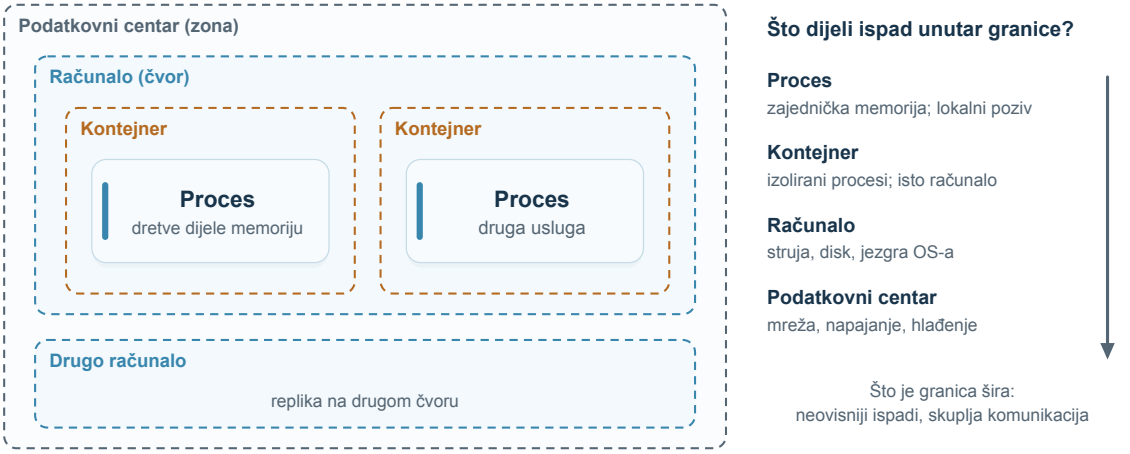



Figure 7: Granice razmještaja su ugniježđene. Što je granica šira, komponente s njezine dvije strane ispadaju neovisnije, ali je komunikacija među njima skuplja.

Svaka razina ima drukčija svojstva:

| Granica | Što unutar nje dijeli ispad | Komunikacija preko granice |
|---|---|---|
| Proces | memorija i dretve; pogreška može srušiti cijeli proces | lokalni poziv unutar; poruka između procesa |
| Kontejner | proces je izoliran, ali dijeli jezgru operacijskog sustava i računalo | poruka preko lokalne ili virtualne mreže |
| Računalo (čvor) | napajanje, disk, mrežna kartica, operacijski sustav | poruka preko lokalne mreže |
| Podatkovni centar | napajanje, hlađenje i vanjska mreža | poruka preko javne mreže, veća latencija |

Tri replike provjere u tri kontejnera na istom računalu štite nas od pada jednog procesa, ali ne od pada računala. Za tu zaštitu replike moraju biti na različitim računalima.

## 9.1 Primjer: Compose i Swarm

Isti skup usluga možemo pokrenuti na dva načina. U našem primjeru Docker Compose pokreće sve kontejnere na jednom računalu. Docker Swarm raspoređuje replike usluge na više čvorova. Kada čvor postane nedostupan, Swarm raspoređuje njegove zadatke na druge čvorove kako bi održao zadani broj replika [7].

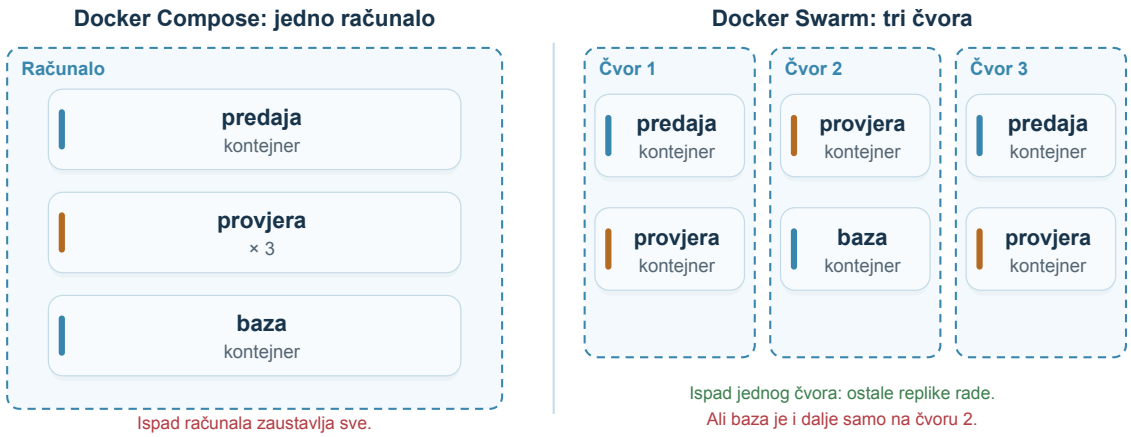



Figure 8: Iste usluge na jednom računalu i na tri čvora. Na jednom računalu ispad računala zaustavlja sve. Na tri čvora replike predaje i provjere preživljavaju ispad jednog čvora, ali baza je i dalje samo na jednom čvoru.

Logička arhitektura je ista, ali se granice ispada razlikuju. Swarm ne rješava sve: ako baza drži podatke na lokalnom disku čvora 2, novi kontejner baze na drugom čvoru neće imati te podatke. Za to su potrebne kopije podataka, odnosno replikacija.

## 10 Natrag na početak: kamo s provjerom?

Sada možemo obrazložiti odgovor. Provjera projekta zaslužuje zasebnu uslugu jer:

- treba drukčije resurse i drukčiji broj replika od ostatka portala;
- njezina pogreška ne smije srušiti primanje predaja;
- može se izdavati neovisno, na primjer kada se mijenja biblioteka za raspakiravanje.

Granica ima cijenu. Predaja i provjera sada razmjenjuju poruke. Portal mora razlikovati stanja **primljeno** i **provjereno** te studentu prikazati koje je od njih postignuto. Ako provjera ne radi, predaje bi se i dalje trebale primati i provjeriti kasnije. To znači da usluga predaje ne treba čekati sinkroni odgovor provjere, nego može poslati posao u red (predavanje 10). Odgovor „ovisi” bio je, dakle, točan, a sada znamo o čemu ovisi.

To je ista ideja kao zadatak s kraja predavanja 01: prijenos završi u 23:59, a provjere u 00:19. Ako sustav zapamti trenutak primanja kao dokaz, student može znati da je predao na vrijeme i prije nego što provjera završi.

## 11 Kako napraviti lošu granicu

Granice usluga lako je pogrešno povući. Česti su sljedeći obrasci:

- usluga za svaku tablicu ili sitnu funkciju, pa jedan korisnički zahtjev prolazi kroz desetak usluga;
- zajednička baza u koju sve usluge pišu;
- usluge koje se moraju izdavati zajedno;
- razgovorljiva sučelja sa stotinama poziva za jedan prikaz;
- dugi sinkroni lanci bez isteka čekanja;
- podjela „po tehnologiji” (sučelje, pozadina, baza) predstavljena kao podjela na usluge.

Svaki prijelaz granice mora se isplatiti: izolacijom ispada, zasebnim skaliranjem ili neovisnim razvojem.

# 12 Zadaci i rasprava

1. **Podjela portala.** Portal ima funkcije: prijava korisnika, predaja, rokovi i pravila zadataka, provjera projekta, pohrana datoteka, obavijesti nastavniku i ocjenjivanje. Predložite usluge. Za svaku granicu navedite koja poruka je prelazi, treba li sinkroni odgovor i što se događa ako druga strana ne radi.
2. **Dostupnost lanca.** Zahtjev ovisi o četiri usluge, svaka dostupna 99,5 % vremena, s neovisnim ispadima. Kolika je dostupnost lanca i koliko je to sati nedostupnosti godišnje? Što se mijenja ako sve četiri rade na istom računalu?
3. **Razgovorljivo sučelje.** Prikaz rezultata ispita šalje 300 uzastopnih zahtjeva usluzi korisnika, svaki traje 1,5 ms. Koliko traje samo čekanje na te odgovore? Predložite dvije izmjene i navedite kompromis svake od njih.
4. **P2P ili poslužitelj?** Za svaki slučaj odaberite klijent–poslužitelj ili P2P i obrazložite: (a) distribucija skupa podataka od 50 GB računalima u laboratoriju; (b) predaja ispita s dokazom vremena predaje; (c) dijeljenje snimki predavanja među studentima.
5. **Granice razmještaja.** U Compose datoteci pokrećete tri replike provjere na jednom računalu. Od kojih ispada vas to štiti, a od kojih ne? Što trebate promijeniti da sustav preživi ispad računala?
6. **Klaster.** U Spark aplikaciji jedan executor se sruši usred obrade. Drugi put se sruši driver. Što od toga očekujete u svakom slučaju i zašto je razlika važna?

## 12.1 Predloženo rješenje

1. Razumna podjela su usluge prijave, predaje (s pohranom zapisa), rokova, provjere i obavijesti. Predaja sinkrono pita prijavu i rokove jer bez njih ne može odlučiti smije li primiti predaju. Provjeri i obavijestima šalje asinkrone poruke jer studentu treba potvrda primanja, a ne završena provjera ni poslana e-pošta. Ako rokovi ne rade, predaja mora odbiti ili odgoditi odluku; ako obavijesti ne rade, predaja i dalje uspijeva. Ocjenjivanje i pohrana mogu biti zasebne usluge ili dio predaje; obje odluke treba obrazložiti.
2. $0{,}995^4 \approx 0{,}980$, oko 2 % vremena ili približno 174 sata godišnje. Ako sve četiri rade na istom računalu, ispadi nisu neovisni: ispad računala ruši sve odjednom pa formula više ne opisuje sustav.

3. $300 \times 1{,}5$ ms $= 450$ ms. Jedan zahtjev za svih 300 korisnika smanjuje broj poruka, ali traži novu operaciju u sučelju usluge. Lokalna kopija imena uklanja pozive, ali može zastarjeti.
4. (a) P2P je prikladan jer računala u laboratoriju mogu dijeliti dijelove, a sažeci štite od oštećenih dijelova. (b) Klijent–poslužitelj jer je potrebno pouzdano mjesto koje bilježi vrijeme i čuva predaju. (c) Oba su moguća; P2P štedi poslužitelj, ali snimka je dostupna samo dok je netko nudi.
5. Štiti od pada jednog procesa ili kontejnera, ne od pada računala, njegova diska ni napajanja. Replike treba rasporediti na više računala, na primjer kao uslugu u Swarmu s više čvorova.
6. Ako se sruši executor, driver može njegove zadatke ponovno dodijeliti drugim executorima i obrada se nastavlja. Ako se sruši driver, nestaje proces koji zna stanje cijele aplikacije pa je aplikaciju potrebno ponovno pokrenuti. Razlika pokazuje da je koordinator posebno osjetljiv dio arhitekture.

# 13 Sažetak

- Arhitektura opisuje komponente, odgovornosti i veze, ali i smještaj komponenti; ista logička podjela s drukčijim smještajem ima drukčija svojstva.
- Klijent–poslužitelj drži stanje i pravila na poznatom mjestu, ali klijenti ovise o poslužitelju. Slojeve možemo smjestiti na jedno, dva ili više računala.
- Vertikalna raspodjela dijeli po funkciji, a horizontalna pokreće replike iste funkcije.
- Usluga ima vlastito sučelje i podatke. Svaki prijelaz granice usluge dodaje latenciju, smanjuje dostupnost lanca i pravila preko granice čini težima.
- P2P sustavi skaliraju se s brojem sudionika, ali plaćaju izmjenom sudionika, nepovjerenjem i težim pronalaženjem podataka.
- Granice razmještaja određuju što ispada zajedno. Replike štite od ispada samo ako su izvan granice koja ispada.

## 13.1 Osmislite sljedeći korak: obavijesti nastavniku

Nastavnik želi dobiti e-poštu kada stigne predaja. Usluga za slanje e-pošte ponekad satima ne radi. Gdje biste povukli granicu? Mora li predaja čekati da se e-pošta pošalje? Što ako se ista obavijest pošalje dvaput?

# 14 Izvori i dodatna literatura


1. M. van Steen i A. S. Tanenbaum, *Distributed Systems*, 4. izd., 2023., pogl. 2 „Architectures”. https://www.distributed-systems.net/index.php/books/ds4/
2. J. Lewis i M. Fowler, „Microservices”, 25. 3. 2014. https://martinfowler.com/articles/microservices.html

3. B. Cohen, „The BitTorrent Protocol Specification”, BEP 3. https://www.bittorrent.org/beps/bep_0003.html
4. A. Loewenstern i A. Norberg, „DHT Protocol”, BEP 5. https://www.bittorrent.org/beps/bep_0005.html
5. Apache Spark, „Cluster Mode Overview”. https://spark.apache.org/docs/latest/cluster-overview.html
6. J. Dean, „Designs, Lessons and Advice from Building Large Distributed Systems”, LADIS 2009, slajd „Numbers Everyone Should Know”. https://www.cs.cornell.edu/projects/ladis2009/talks/dean-keynote-ladis2009.pdf
7. Docker, „Swarm mode key concepts”. https://docs.docker.com/engine/swarm/key-concepts/